데이터 분석을 위한 **Python** 이해

- 종합실습 1 -

캠퍼스 : 판교

반 : 9반

이름 : 김민정

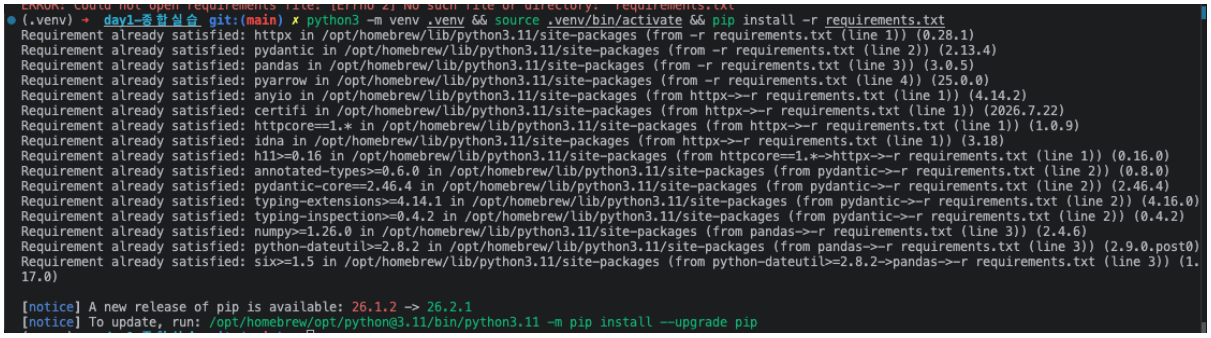
번호 : P282

제출일 : 2026-08-06

실행 결과

1. 가상환경(venv) 생성 및 패키지 설치

python3 -m venv .venv && source .venv/bin/activate && pip install -r requirements.txt



2. 비동기 수집 ⇒ 스키마 검증 - > CSV/Parquet 저장 파이프라인 실행

python 판교_9반_김민정.py

```
(.venv) ➜  day1-종합실습 git:(main) ✗ python 판교_9반_김민정.py

INFO: 요청 시작: open_meteo
INFO: 요청 시작: countries_dev
INFO: 요청 시작: ip_api
INFO: HTTP Request: GET http://ip-api.com/json/8.8.8.8 "HTTP/1.1 200 OK"
INFO: 요청 완료: ip_api (status=200, 0.425초)
INFO: HTTP Request: GET https://countries.dev/alpha/KOR "HTTP/1.1 200 OK"
INFO: 요청 완료: countries_dev (status=200, 0.862초)
INFO: HTTP Request: GET https://api.open-meteo.com/v1/forecast?latitude=37.5665&longitude=126.9780&hourly=temperature_2m,preci
ays=3&timezone=Asia/Seoul "HTTP/1.1 200 OK"
INFO: 요청 완료: open_meteo (status=200, 1.002초)

비동기 수집 완료 — 3개 API, 총 1.030초 소요
  - open_meteo: 성공 (1.002초)
  - countries_dev: 성공 (0.862초)
  - ip_api: 성공 (0.425초)
결과 저장: /Users/minjeong/WebProject/python.0806/day1-종합실습/async_api_result.json
INFO: open_meteo 스키마 검증 성공 (72건)
INFO: countries_dev 스키마 검증 성공
INFO: ip_api 스키마 검증 성공

스키마 검증 결과
  - open_meteo: 통과 (72건)
  - countries_dev: 통과
  - ip_api: 통과
검증 결과 저장: /Users/minjeong/WebProject/python.0806/day1-종합실습/validated_result.json (0.002초)
INFO: open_meteo 성능: 쓰기 CSV 0.0015초/Parquet 0.0014초, 읽기 CSV 0.0006초/Parquet 0.0011초 (72행)
INFO: countries_dev 성능: 쓰기 CSV 0.0005초/Parquet 0.0005초, 읽기 CSV 0.0003초/Parquet 0.0006초 (1행)
INFO: ip_api 성능: 쓰기 CSV 0.0002초/Parquet 0.0005초, 읽기 CSV 0.0003초/Parquet 0.0005초 (1행)

CSV vs Parquet 읽기/쓰기 성능 비교
  - open_meteo (72행)
      쓰기: CSV 0.0015초 / Parquet 0.0014초 -> Parquet가 1.1배 빠름
      읽기: CSV 0.0006초 / Parquet 0.0011초 -> CSV가 2.0배 빠름
  - countries_dev (1행)
      쓰기: CSV 0.0005초 / Parquet 0.0005초 -> CSV가 1.1배 빠름
      읽기: CSV 0.0003초 / Parquet 0.0006초 -> CSV가 2.0배 빠름
  - ip_api (1행)
      쓰기: CSV 0.0002초 / Parquet 0.0005초 -> CSV가 1.9배 빠름
      읽기: CSV 0.0003초 / Parquet 0.0005초 -> CSV가 2.0배 빠름

전체 파이프라인 소요 시간: 1.159초 (수집 1.030초 + 검증 0.002초 + 저장·읽기 0.008초)
```

3. pytest 실행 결과

python -m pytest –v

```
(.venv) ➜  day1-종합실습 git:(main) ✗ python -m pytest -v
============================= test session starts ==============================
platform darwin -- Python 3.11.15, pytest-9.1.1, pluggy-1.6.0 -- /opt/homebrew/opt/python@3.11/bin/python3.11
cachedir: .pytest_cache
rootdir: /Users/minjeong/WebProject/python.0806/day1-종합실습
plugins: anyio-4.14.2
collected 14 items

test_판교_9반_김민정.py::test_weather_record_valid PASSED                [  7%]
test_판교_9반_김민정.py::test_weather_record_precipitation_out_of_range PASSED [ 14%]
test_판교_9반_김민정.py::test_weather_record_invalid_temperature_type PASSED [ 21%]
test_판교_9반_김민정.py::test_country_info_valid PASSED                  [ 28%]
test_판교_9반_김민정.py::test_country_info_negative_population PASSED    [ 35%]
test_판교_9반_김민정.py::test_country_info_alpha3code_wrong_length PASSED [ 42%]
test_판교_9반_김민정.py::test_country_info_population_invalid_type PASSED [ 50%]
test_판교_9반_김민정.py::test_ip_info_valid PASSED                       [ 57%]
test_판교_9반_김민정.py::test_ip_info_latitude_out_of_range PASSED       [ 64%]
test_판교_9반_김민정.py::test_ip_info_country_code_wrong_length PASSED   [ 71%]
test_판교_9반_김민정.py::test_validate_results_all_valid PASSED          [ 78%]
test_판교_9반_김민정.py::test_validate_results_missing_field_returns_none PASSED [ 85%]
test_판교_9반_김민정.py::test_validate_results_type_error_returns_none PASSED [ 92%]
test_판교_9반_김민정.py::test_validate_results_out_of_range_returns_none PASSED [100%]

============================== 14 passed in 0.38s ==============================
```

4. ruff 코드 스타일 검사 결과

ruff check .

```
(.venv) ➜  day1-종합실습 git:(main) ✗ ruff check .
All checks passed!
```

5. Git 커밋 이력

git log --oneline –5

```
1bc35de (HEAD -> main) feat: pytest로 스키마 검증 테스트 작성
ee92a00 (origin/main, origin/HEAD) feat: CSV 및 Parquet 읽기/쓰기 성능 측정 기능 추가
eb850e0 feat: CSV vs Parquet 읽기/쓰기 성능 비교
b67773b feat: api 결과에 대한 추가 Pydantic v2 validation
100fdfd chore: 폴더구조 정리
(END)
```

성능 비교 요약

데이터셋	행 수	쓰기 우세	읽기 우세
open_meteo	72행	Parquet (1.4배)	CSV (1.5배)
countries_dev	1행	CSV (1.7배)	CSV (2.0배)
ip_api	1행	CSV (2.0배)	CSV (2.1배)

행 수가 매우 적은 countries_dev, ip_api는 CSV/Parquet 차이가 사실상 오차 범위(0.1~0.4ms)이고, 상대적으로 행이 많은 open_meteo(72행)에서도 아직 파일 크기가 작아 두 포맷의 차이가 뚜렷하게 드러나지는 않았습니다. Parquet의 이점(압축, 컬럼 스토리지, 스키마 보존)은 데이터 규모가 훨씬 커야 명확해질 것으로 보입니다.

코드에 대한 의견

4-1. Pydantic 검증 시 필드를 하나씩 꺼내서 넘긴 이유

처음엔 CountryInfo.model_validate(cd)처럼 dict를 통째로 넘기면 코드가 더 짧아지겠다

고 생각했습니다. 그런데 그렇게 하면 API 응답에 있는 모든 필드가 그대로 들어가 버려서, 과제에서 요구하는 "필요한 필드만 추출"이 코드상으로 잘 드러나지 않을 것 같았습니다. 그래서 조금 더 길더라도 필드를 하나씩 꺼내서 넘기는 지금 방식으로 작성했습니다.

4-2. Parquet 읽기 성능 측정값이 들쭉날쭉했던 이유와 대응

여러 번 돌려보니 open_meteo의 Parquet 읽기 시간이 어떤 실행에서는 0.03초, 어떤 실행에서는 1.7초까지 널뛰었습니다. 처음엔 "데이터가 커서 그런가?" 싶었는데, 같은 파일을 같은 프로세스에서 두 번 읽어보니 첫 번째만 느리고 두 번째부터는 0.001초 이하로 뚝 떨어지는 것을 확인했습니다. 즉 프로세스에서 pyarrow 엔진을 처음 쓸 때 드는 초기화 비용이 측정값에 섞여 들어간 것이었습니다. 이를 그대로 두면 "CSV가 Parquet보다 압도적으로 빠르다"는 잘못된 결론이 날 수 있어서, 실제 측정 전에 더미 데이터로 Parquet를 한 번 써보고 읽어 엔진을 미리 예열시키고, 그 비용은 측정에서 제외하도록 했습니다.

4-3. asyncio.run()을 매번 새로 여는 구조에 대한 고민

지금은 스크립트를 한 번 실행하고 끝나는 구조라 asyncio.run()으로 이벤트 루프를 매번 새로 여는 것이 문제없다고 판단했습니다. 다만 이걸 스케줄러에 올려 주기적으로 반복 수집하게 된다면, 매번 AsyncClient를 새로 만드는 지금 구조는 비효율적일 수 있어 그때는 클라이언트를 재사용하는 방향으로 바꿔야 한다고 생각합니다. 지금 과제 범위에서는 그렇게까지 할 필요는 없다고 판단해 현재 구조로 남겨두었습니다.

4-4. 예외 처리 분기가 실제로 실행되는 걸 확인하지 못한 점

예외 처리 코드를 짜면서 "이게 실제로 실패하는 경우를 본 적이 없는데 제대로 동작하는 게 맞나?"라는 생각이 들었습니다. 지금은 API들이 다 정상 응답을 주기 때문에 KeyError/TypeError/ValidationError 분기가 실행되는 것을 실제 API 호출로는 확인하지 못했습니다. (다만 test_판교_9반_김민정.py의 test_validate_results_missing_field_returns_none, test_validate_results_type_error_returns_none,
test_validate_results_out_of_range_returns_none 세 테스트로 이 분기들이 의도대로 동작하는 것은 별도로 검증했습니다.) 실제 API 실패 상황까지 재현하려면 respx 같은 라이브러리로 HTTP 응답 자체를 가짜로 만들어 필드 누락·잘못된 타입을 강제로 주입하는 테스트가 추가로 필요하다고 생각합니다.

추가 의견 (개선 사항 / 코드 품질 개선 측면)

- respx로 HTTP 레벨 실패 케이스 테스트 추가 — 현재 테스트는 validate_results()에 정상/비정상 dict를 직접 넣어 검증 로직만 테스트합니다. respx나 pytest-httpx로 fetch_json/fetch_all이 실제로 404·타임아웃·잘못된 JSON을 받았을 때 어떻게 동작하는지까지 테스트에 포함하면 커버리지가 더 완전해질 것 같습니다.

- 타임아웃/재시도 정책 명시화 — 현재 timeout=10.0으로 고정되어 있는데, 실제 배포 환경에서는 API별로 타임아웃을 다르게 주거나 httpx의 Retry 정책을 붙이는 것이 안정성 측면에서 도움이 될 것 같습니다.

- 성능 비교의 데이터 규모 한계 — 지금은 최대 72행 수준이라 CSV와 Parquet의 실질적 차이(압축률, 대용량에서의 I/O 이득)를 체감하기 어렵습니다. 기회가 된다면 수천~수만 행 규모의 합성 데이터로 같은 비교를 반복해 포맷별 장단점을 더 설득력 있게 보여줄 수 있을 것 같습니다.